CLean: Lean Semantics, PTX Ingestion, and the Saxpy Proof Path

Architecture, methodology, implementation, and remaining proof obligations

Riyaza

Technical presentation for PhD advisor

May 14, 2026

Deck Roadmap

1. High-level research idea and design goals.
2. Core implementation: types, state, instructions, typing.
3. Semantics: executable helper layer and relational small-step layer.
4. Proof methodology: computation lemmas, frame lemmas, determinism, decomposition.
5. PTX front end: AST, parser, checked lowering, bridge theorem surface.
6. Saxpy case study: PTX input, state construction, correctness pipeline, current blockers.

One-Sentence Thesis

CLean is a Lean 4 framework for giving a proof-facing, executable semantics to a supported subset of PTX-like GPU kernels, then using that semantics to prove concrete and eventually parametric kernel correctness.

Current repo scope

The repo already has a semantic state model, an IR, executable evaluation functions, relational small-step rules, PTX parsing and checked lowering, proof infrastructure, and case studies for toy kernels, saxpy, and matmul.

Why This Is Useful

- GPU kernels are usually validated by testing, compiler inspection, and informal reasoning.
- PTX is low-level enough to expose memory spaces, thread IDs, predicates, branches, barriers, and instruction effects.
- Lean lets us state exactly what a parsed-and-lowered kernel computes.
- Executable semantics let small concrete cases run by `native_decide`; relational semantics support proof statements over arbitrary traces.

Design Principle

The project avoids building a “PTX text interpreter” as the trusted core.

Instead:

- parse raw PTX text into a syntax AST,
- checked-lower that AST into a smaller semantic IR,
- execute the IR against a concrete state model,
- prove properties of the IR semantics,
- expose parse/lower soundness as the trusted boundary.

Repo Topology

`CLean/Core`	scalar types, values, addresses, IR, state, typing
`CLean/Semantics`	helper evaluator, small-step relation, executable runner
`CLean/Proof`	frame lemmas, computation lemmas, loop API, determinism, decomposition
`CLean/PTX`	PTX AST, parser, lowering, checked bridge
`CLean/Examples`	semantic smoke tests, parser/lowering tests, saxpy, matmul

Status Legend

proved	Lean proof has no local `sorry`.
executable	executable function or concrete example checked by `native_decide`.
admitted	theorem statement exists, but body contains `sorry`; this is an explicit proof obligation.
future work	concept is architected but not implemented or not yet proof-complete.

The deck uses these statuses to separate implemented machinery from admitted proof surface.

Main Architectural Pipeline

PTX text $\rightarrow$ `PTX.Ast` $\rightarrow$ checked lowering $\rightarrow$ CLean IR $\rightarrow$ small-step semantics $\rightarrow$ kernel postcondition

The core proofs are intended to live after lowering, where the instruction set is smaller and semantic cases are explicit.

Methodological Split

- **Executable layer:** functions returning `Option State`, used for simulation, tracing, and concrete regression tests.
- **Relational layer:** inductive propositions `StepInstr`, `StepBlock`, `StepWarp`, `StepMachine`.
- **Bridge layer:** soundness theorems from executable steps to relational steps.
- **Correctness layer:** `PartialCorrect`, `TotalCorrect`, loop specs, and kernel-specific postconditions.

Implemented Surface Area

- Scalar integer, predicate, bitwise, address conversion, load/store, branch, barrier, and simple float operations are represented.
- Warp-level operations, atomics, and MMA are present in the IR shape, but most are not semantically executed yet.
- PTX parser supports declarations, labels, predicated branches, params, shared declarations, memory operations, arithmetic, comparisons, and unsupported-instruction reporting.
- Examples validate end-to-end parser, lowering, and execution on small kernels.

Current Proof Surface

- proved: executable-step soundness into `StepMachine`.
- proved: runN reachability and stuck-stability theorems.
- proved: single-warp determinism bridge, modulo preservation lemmas used beneath it.
- admitted: several frame and preservation lemmas in `Proof/Lemmas.lean`.
- admitted: parser/lowering soundness theorem surfaces in `Proof/Bridge.lean`.
- admitted: parametric saxpy symbolic execution lemma.

Advisor-Level Takeaway

The project has moved from design sketch to an executable, typed, PTX-ingesting semantic core.

The remaining work is not a vague "prove everything" task. It decomposes into:

- finish frame/preservation lemmas,
- complete lane-disjointness/decomposition,
- do straight-line symbolic execution for saxpy,
- package parser/lowering soundness,
- then scale to loop kernels like matmul.

Core Files

`Core/Types.lean`	scalar types, values, address spaces, special registers
`Core/IR.lean`	semantic instruction vocabulary
`Core/State.lean`	machine state, lane/warp/CTA/global memory
`Core/Typing.lean`	computable type signatures and memory preconditions

Scalar Type Universe

- `ScalarTy` includes predicates, unsigned/signed integers, bit-vectors, half/bfloat/f32/f64.
- The semantic layer uses scalar tags directly rather than encoding everything as raw bitvectors.
- This keeps type-directed memory encoding and instruction support explicit.
- Unsupported or not-yet-modeled cases can fail by returning `none` rather than silently inventing behavior.

Scalar Types in Lean

```
inductive ScalarTy where
  | pred
  | u8 | u16 | u32 | u64
  | s8 | s16 | s32 | s64
  | b8 | b16 | b32 | b64
  | f16 | bf16 | f32 | f64
  deriving Repr, DecidableEq, Inhabited
```

This is intentionally close to PTX scalar suffixes: `.u32`, `.s32`, `.b64`, `.f32`, etc.

Address Spaces

- PTX state spaces are not erased into one integer address space.
- `AddrSpace` tracks `global`, `shared`, `local`, `param`, `const`, and `generic`.
- Concrete `Addr` constructors carry the scope needed for shared and local memory.
- Generic addresses carry a tag and offset, letting `cvta` and `isspacep` have semantic content.

Address Model

```
inductive Addr where
  | global (offset : Nat)
  | shared (cta : CTAId) (offset : Nat)
  | local (cta : CTAId) (warp : WarpId) (lane : LaneId) (offset : Nat)
  | param (offset : Nat)
  | const (offset : Nat)
  | generic (space : AddrSpace) (offset : Nat)
```

This is one of the most important proof choices: state-space identity is part of the address.

Value Model

- `Value` mirrors scalar types and includes semantic addresses.
- Signed integer values are stored as Lean `Int`, with normalization helpers for fixed-width behavior.
- Floating values are represented using Lean `Float`; this is convenient but not bit-exact PTX f32 semantics.
- Tensor fragments are represented by `Value.frag`, but MMA execution is not yet implemented.

Selected Value Constructors

```
inductive Value where
  | pred (b : Bool)
  | u32 (x : UInt32)
  | u64 (x : UInt64)
  | s32 (x : Int)
  | s64 (x : Int)
  | f32 (x : Float)
  | f64 (x : Float)
  | gaddr (space : AddrSpace) (offset : Nat)
  | frag (ty : FragTy) (payload : Array Value)
```

Thread and Grid Identity

- `CTAId` and `WarpId` are natural numbers.
- `LaneId := Fin 32`, so lane identity is bounded at the type level.
- `GridCtx` stores `gridDim` and `blockDim`.
- Helpers compute `tid.x`, `ctaid.x`, `ntid.x`, and related special registers from this context.

Memory Model

- Every memory space is modeled as a byte map: `Std.HashMap Nat UInt8`.
- Scalar loads/stores encode and decode little-endian byte sequences.
- Typed memory access requires codec support, byte width, alignment, and address-space agreement.
- This gives precise failure points for unsupported or ill-typed accesses.

Top-Level State

```
structure State where
  kernelEnv : KernelEnv := {}
  global : GlobalMem := {}
  const : ConstMem := {}
  param : ParamMem := {}
  ctas : Std.HashMap CTAId CTAState := {}
  atomics : AtomicState := {}
```

The top-level state separates immutable-ish launch environment from mutable memory and CTA state.

CTA, Warp, Lane

```
structure LaneState where
  regs : Std.HashMap RegName Value := {}
  preds : Std.HashMap PredName Bool := {}
  localMem : LocalMem := {}
  pc : PC := ("entry", 0)
  status : LaneStatus := .running

structure WarpState where
  lanes : Array LaneState := Array.replicate 32 {}
  activeMask : UInt32 := 0xFFFFFFFF
  exitedMask : UInt32 := 0
```

Well-Formedness Checks

- `WarpState.wf?`: lane array size is exactly 32.
- `CTAState.wf?`: every warp in the CTA is well formed.
- `KernelEnv.wf?`: block map keys agree with block labels.
- `State.wf?`: kernel environment is well formed and every CTA is well formed.

Why Boolean and Prop Versions Exist

- Functions like `stepInstr?` need executable booleans.
- Relational rules and proofs need propositions.
- The project provides bridge theorems like `wf_iff_bool`.
- This pattern repeats for `RunnablePc`, `ParticipatingRunnable`, and `lockstepRunnable`.

Instruction Vocabulary

- Pure expression layer: `RValue`.
- Guarded instruction layer: `GInstr`.
- Block layer: array of guarded instructions plus a terminator.
- Effects include register/predicate writes, load/store, address conversion, space tests, barriers, warp ops, atomics, and MMA.

RValue

```
inductive RValue where
  | imm (v : Value)
  | reg (r : RegName)
  | pred (p : PredName)
  | special (s : SpecialReg)
  | unop (op : ScalarUnaryOp) (a : RValue)
  | binop (op : ScalarBinaryOp) (a b : RValue)
  | triop (op : ScalarTernaryOp) (a b c : RValue)
```

This is the pure expression language used by both instructions and branch conditions.

Instruction Shape

```
inductive Instr where
  | assignReg (dst : RegName) (rhs : RValue)
  | assignPred (dst : PredName) (cmp : CmpExpr)
  | assignPredValue (dst : PredName) (rhs : RValue)
  | load (dst : RegName) (src : TypedAddr)
  | store (dst : TypedAddr) (value : RValue)
  | cvta (dst : RegName) (space : AddrSpace) (src : RValue)
  | isspacep (dst : PredName) (space : AddrSpace) (src : RValue)
  | barrierCTA (barrierId : Nat)
  | warp (op : WarpOp)
  | atomic ...
  | mma ...
```

Guarded Instructions

- `GInstr` pairs an optional guard with an instruction.
- A guard is a predicate register plus an optional negation bit.
- Participation is computed lane-by-lane against runnable lanes at the current PC.
- Predicated-off lanes at barriers are explicitly advanced rather than blocked.

Terminators and Blocks

- `Terminator.br`: unconditional branch to a block label.
- `Terminator.cbr`: conditional branch with true and false labels.
- `Terminator.terminate`: marks current runnable lanes as terminated.
- A `Block` is a label, body array, and terminator.

Typing Layer

- `Core/Typing.lean` is the repo's local type checker for the semantic IR and lowering phase.
- It defines byte widths and alignments.
- It computes signatures for unary, binary, ternary, and comparison operations.
- It defines typed memory access preconditions.

Typed Access Preconditions

```
def typedAccessPreconditions? (space : AddrSpace)
    (ty : ScalarTy) (addr : Addr) : Bool :=
  scalarCodecSupported? ty &&
  (byteWidth? ty).isSome &&
  aligned? ty addr &&
  addrSpaceMatches? space addr
```

This check is used by both `readMem?` and `writeMem?`.

Scalar Compatibility

- Checked lowering accepts some PTX-compatible aliases: for example `s32`, `u32`, and `b32`.
- `scalarCompatible?` captures these expected/actual relationships.
- `coerceOperandTo?` inserts conversions when the semantic value needs an explicit shape.
- 64-bit pointer-shaped values are deliberately kept unwrapped in some cases because runtime `cvta` may produce generic addresses.

Supported Operation Signatures

- `unarySig?`: moves, negation, abs, bitnot, conversion.
- `binarySig?`: add, sub, mul, wide multiply, bitwise logic, shifts, min/max.
- `ternarySig?`: `mad`, `fma`, `selp`.
- `cmpSig?`: same-type integer/float comparisons returning predicate.

Core Model Summary

- The state model is concrete enough to execute kernels.
- The IR is small enough to reason about directly.
- The typing layer rejects unsupported or inconsistent PTX before semantics.
- The major abstraction boundary is already in place: PTX syntax lowers into semantic effects.

Semantics Files

`Semantics/Helpers.lean`	executable evaluator and state transformers
`Semantics/SmallStep.lean`	relational small-step rules
`Semantics/Execution.lean`	reachability, step driver, runN, traceN

Executable Helper Layer

- Most semantic content lives in functions ending with `?`.
- Failure is represented by `Option.none`.
- This keeps unsupported cases, bad lookups, invalid alignment, and divergent branch conditions explicit.
- The same functions are used for concrete execution and for proof rules.

Runnable Lanes

- `laneIsRunnable` checks lane status and active mask bit.
- `runnableLaneIds` filters all 32 lanes.
- `currentRunnablePc?` takes the first runnable lane's PC.
- `lockstepRunnable?` requires all runnable lanes to have the same PC.

Participation

- `participatingRunnableLaneIds?` starts from runnable lanes.
- It filters to lanes at the current PC.
- It evaluates the optional guard per lane.
- It returns the lane list that actually participates in the instruction effect.

Special Registers

- `evalSpecial` maps PTX special registers to values.
- Thread IDs come from block-local linear lane identity plus `blockDim`.
- CTA IDs are decoded from a linear `CTAId` and `gridDim`.
- `ntid` and `nctaid` come directly from launch context.

Expression Evaluation

- `evalRValue?` is mutually recursive with scalar operator evaluation.
- Register and predicate reads are lane-local.
- Special registers are computed from `GridCtx`, CTA, warp, and lane.
- Pure operators implement arithmetic, logical, conversion, and address arithmetic behavior.

RValue Evaluation Skeleton

```
partial def evalRValue? st cta warp lane : RValue -> Option Value
  | .imm v => some v
  | .reg r => do
      let laneState <- st.getLane? cta warp lane
      readReg laneState r
  | .pred p => do
      let laneState <- st.getLane? cta warp lane
      let b <- readPred laneState p
      pure (.pred b)
  | .special s => some <| evalSpecial st.kernelEnv.gridCtx cta warp lane s
  | .binop op a b => ...
```

Memory Read and Write

- `readMem?` checks typed access preconditions.
- It finds the correct byte map for the address space.
- It reads the required byte width and decodes the scalar.
- `writeMem?` performs the dual encode/write/update operation.

Address Resolution

- `resolveAddr?` evaluates a `TypedAddr`'s address expression.
- Integer-like offsets lower into concrete state-space addresses.
- Generic addresses resolve only when their tag matches the requested space, except `.generic` keeps the tag.
- Shared and local addresses add CTA, warp, and lane scope.

Address Conversion

- `evalCvta?` maps integer or already-generic pointer-shaped values to `Value.gaddr`.
- `evalIsspacep?` tests the tag of a generic address.
- This supports kernels that load pointers from parameter memory before global memory access.
- Saxpy relies on this path: parameter pointers are loaded, converted to global addresses, offset, and dereferenced.

Instruction Step Function

- `stepInstr?` fetches warp state and checks lockstep.
- It computes participating lanes under the optional guard.
- For register/predicate operations, it applies a lane-local update over participants.
- For stores, it sequentializes participating lanes in lane-list order.
- It then advances runnable PCs, except for barriers which have special handling.

stepInstr? Shape

```
def stepInstr? (st : State) (cta : CTAId) (warp : WarpId)
    (gi : GInstr) : Option State := do
  let warpState <- st.getWarp? cta warp
  if !lockstepRunnable? warpState then none else
    let participants <- participatingRunnableLaneIds? warpState gi.guard?
    match gi.instr with
    | .barrierCTA barrierId => stepBarrierCTA? st cta warp barrierId participants
    | _ => do
        let st <- match gi.instr with
          | .assignReg dst rhs => ...
          | .load dst src => ...
          | .store dst value => ...
          | _ => none
        advanceRunnablePcs? st cta warp
```

Barrier Semantics

- `stepBarrierCTA?` models a CTA barrier instance keyed by barrier id.
- Predicated-off lanes skip the barrier and advance.
- Participating lanes record arrival and become `blockedBarrier`.
- Once arrivals reach expected count, all arrived lanes are released, advanced, and marked running.

Terminator Semantics

- `br`: sets PC of current lanes to target block index 0.
- `terminate`: marks current lanes terminated.
- `cbr`: evaluates condition for all current lanes and requires a uniform destination.
- Non-uniform branch conditions currently fail by returning `none`.

Relational Small-Step Layer

- `StepInstr` wraps `stepInstr?` with preconditions.
- `StepBlock` chooses between body instruction and block terminator.
- `StepWarp` packages a block step for a CTA/warp.
- `StepMachine` is the global one-step relation.

StepInstr Rule

```
inductive StepInstr : State -> CTAId -> WarpId -> GInstr -> State -> Prop where
  | mk :
      State.wf st ->
      st.getWarp? cta warp = some warpState ->
      WarpState.wf warpState ->
      Helpers.lockstepRunnable warpState ->
      Helpers.ParticipatingRunnable warpState gi.guard? participants ->
      Helpers.stepInstr? st cta warp gi = some st' ->
      StepInstr st cta warp gi st'
```

Executable Driver

- `currentInstrStep?`: attempt the current body instruction.
- `currentTermStep?`: attempt the terminator when body index is past the body.
- `stepAt?`: instruction first, then terminator.
- `step?`: default scheduler for CTA 0, warp 0.

runN and traceN

- `runN fuel st` executes up to `fuel` successful `step?` transitions.
- If `step?` returns `none`, `runN` stays at the current state.
- `runN?` fails if the requested number of steps cannot be taken.
- `traceN` records intermediate states for debugging and smoke tests.

Reachability

- `Reaches` is the reflexive-transitive closure of `StepMachine`.
- `StepMachine.runN_reaches`: every executable run is a relational reachability trace.
- `StepMachine.runN?_sound`: successful exact-fuel execution is also relationally reachable.
- This is the base connection between executable testing and proof-facing semantics.

Semantics Summary

The semantics now has two synchronized views:

- a computable interpreter for concrete kernels;
- an inductive step relation for theorem statements.

The key engineering challenge is keeping these two views aligned without duplicating semantic definitions.

Proof Files

`Proof/Lemmas.lean`	frame, memory, wf, executable-to-relational helpers
`Proof/InstrCompute.lean`	per-instruction computation over lane lists
`Proof/Loops.lean`	correctness definitions and counted-loop specs
`Proof/Determinism.lean`	single-warp determinism bridge
`Proof/IsSingleWarpPres.lean`	preservation of single-warp support
`Proof/LaneDecomposition.lean`	lane-local view and lifting architecture
`Proof/Bridge.lean`	parser/lowering soundness theorem surface

Correctness Vocabulary

- `MachineFinal st`: no `StepMachine` successor exists.
- `TerminatesAt init final`: `Reaches init final` and `MachineFinal final`.
- `PartialCorrect init post`: every terminal final state satisfies `post`.
- `TotalCorrect init post`: some terminal final state exists and satisfies `post`.

Correctness Definitions

```
def MachineFinal (st : State) : Prop :=
  forall st', not (StepMachine st st')

def TerminatesAt (init final : State) : Prop :=
  Reaches init final /\ MachineFinal final

def PartialCorrect (init : State) (post : State -> Prop) : Prop :=
  forall final, TerminatesAt init final -> post final
```

Computation Lemma Pattern

- First prove structural unfoldings of executable loops.
- Then prove one-lane or list-of-lanes effects.
- Then package these effects as instruction-specific computation lemmas.
- This is especially important because `stepInstr?` uses imperative-style `for` loops inside `Option`.

applyToLaneIds? Foundation

- `applyToLaneIds?_nil`: empty lane list returns the input state.
- `applyToLaneIds?_cons`: exposes one state update plus recursion.
- `applyToLaneIds?_singleton`: common one-lane simplification.
- `applyToLaneIds?_lane_in`: for nodup lists, participating lane gets the computed state.

Frame Lemma Pattern

- Register writes preserve predicate files, local memory, PC, and status.
- Predicate writes preserve register files, local memory, PC, and status.
- Lane updates preserve other lanes.
- CTA/warp/lane state updates preserve top-level global/const/param fields.

Memory Lemmas

- `natToBytesLE_length`: encoded byte list has requested width.
- `readBytes?_writeBytes_same`: writing bytes then reading same range returns those bytes.
- `decode_encode_*`: scalar roundtrip lemmas for supported types.
- `readMem_writeMem_same_global_*`: typed global write/read roundtrips.

State Preservation Lemmas

- Many instructions should not mutate global/param/const memory.
- Store should preserve lane register and predicate files.
- Barriers should preserve data memory and register/predicate files, but mutate barrier and lane status/PC.
- These facts become reusable frame facts for kernel proofs.

Current Lemma Status

- proved: basic register/predicate read-after-write and non-alias frame lemmas.
- proved: byte encoding, reading, and global memory read-after-write for key scalar types.
- proved: non-store instruction top-memory preservation for several cases.
- admitted: barrier frame lemmas, several reg/pred preservation lemmas, generic wf preservation, and kernelEnv preservation.

Executable-to-Relational Soundness

- `body_of_stepInstr?`: if helper instruction step succeeds under preconditions, produce `StepMachine`.
- `term_of_stepTerminator?`: same for terminators.
- `body_of_currentInstrStep?_computed`: executable current instruction step is sound.
- `term_of_currentTermStep?_computed`: executable current terminator step is sound.

Automation Layer

- `cstep`: discharges concrete `StepMachine` goals by using current-step soundness and `native_decide`.
- `crun`: uses `runN_reaches`.
- `cpost`: calls `native_decide` on boolean postconditions.
- `cfinish`: tries one of the above.

Why Determinism Is Scoped

- General GPU scheduling is intentionally not solved yet.
- The current executable driver steps CTA 0, warp 0.
- Many examples, including saxpy, use a single populated warp.
- The project proves that, under `IsSingleWarp`, any relational step is captured by `step?`.

IsSingleWarp

```
def IsSingleWarp (st : State) : Prop :=
  forall cta warp ws,
    st.getWarp? cta warp = some ws -> cta = 0 /\ warp = 0
```

This predicate says the only warp that exists in the state is CTA 0, warp 0.

Single-Warp Determinism Theorem

- `step?_of_StepMachine`: under `IsSingleWarp`, any `StepMachine st st'` implies `step? st = some st'`.
- `reaches_imp_runN_of_IsSingleWarp`: any reachable final state is some `runN fuel init`.
- `terminal_eq_runN`: terminal states reduce to executable runs.
- This is the core bridge used by saxpy partial correctness.

Stuck Stability

- `step?_eq_none_of_MachineFinal`: final relational state is stuck for the executable scheduler.
- `runN_eq_of_step?_none`: once stuck, extra fuel does nothing.
- `runN_add`: executable runs compose over fuel.
- `runN_eq_of_both_MachineFinal`: two final `runN` endpoints from the same init are equal.

Single-Warp Preservation

- `Proof/IsSingleWarpPres.lean` proves updates do not introduce new warp keys.
- The main invariant is `WarpSupportEq`: two states agree on which CTA/warp lookups succeed.
- Primitive updates such as `setLane`, `writeMem?`, and `advanceRunnablePcs?` preserve support.
- `step?_preserves_IsSingleWarp` is the headline preservation theorem.

Single-Warp Preservation Status

- proved: most primitive support-preservation lemmas.
- proved: instruction, terminator, and `step?` preservation structure.
- admitted: `stepBarrierCTA?_preserves_warp_support` is still a `sorry`.
- Consequence: the high-level preservation theorem currently depends on that admitted barrier lemma.

Loop Proof API

- `CountedLoopSpec` abstracts an invariant, guard, body relation, variant, and postcondition.
- `partial_correct`: invariant preservation plus exit rule implies postcondition.
- `total_correct`: decreasing natural variant gives termination and postcondition.
- This is mostly for future loop-heavy kernels like matmul; saxpy is straight-line after the early-exit branch.

Lane Decomposition Goal

- GPU warp semantics updates multiple lanes at once.
- Saxpy correctness is naturally per element, hence per lane.
- Need to show each lane's local view evolves as expected under full-warp execution.
- Need to combine per-lane facts into a global memory postcondition.

Lane View

- `LaneView` projects one lane's registers, predicates, local memory, PC, and status.
- `LaneLocal st lane` extracts the view at CTA 0, warp 0.
- `instrWriteSet?` computes addresses written by a lane for store/atomic instructions.
- `DisjointLaneWrites` states pairwise write-set disjointness across active lanes.

Lane Decomposition Status

- proved: definitions for write sets, disjointness, lane view, and lift are implemented.
- proved: current commutation statements are weak/existence-shaped and do not yet prove saxpy.
- future work: strengthen `lanes_independent_runN` to a real per-lane run theorem.
- future work: connect lane-disjoint stores to global vector postconditions.

Proof Methodology Summary

- Keep executable semantics as the single semantic source.
- Prove executable steps are sound relational steps.
- Use frame lemmas to localize instruction effects.
- Use determinism for single-warp kernels to reduce arbitrary terminal traces to `runN`.
- Use lane decomposition to lift per-lane symbolic execution to full-warp vector correctness.

PTX Files

`PTX/Ast.lean`	syntax AST independent of semantic IR
`PTX/Parser.lean`	parser combinators with source-position errors
`PTX/Lowering.lean`	checked lowering to CLean IR
`PTX/Bridge.lean`	parse-plus-lower API and error type
`Proof/Bridge.lean`	intended soundness theorem surface

PTX AST Role

- It preserves PTX syntactic distinctions before semantic decisions.
- Operands include registers, predicates, symbols, addresses, immediates, and special registers.
- Instructions include supported operations and an `unsupported` fallback.
- Kernels store declarations, params, shared declarations, and parsed blocks.

PTX Instruction AST

```
inductive Instr where
  | mov (ty : ScalarTy) (dst : RegName) (src : Operand)
  | binop (op : ScalarBinaryOp) (ty : ScalarTy) (dst : RegName) (lhs rhs : Operand)
  | setp (op : CmpOp) (ty : ScalarTy) (dst : PredName) (lhs rhs : Operand)
  | ld (space : AddrSpace) (ty : ScalarTy) (dst : RegName) (addr : Operand)
  | st (space : AddrSpace) (ty : ScalarTy) (addr value : Operand)
  | cvta (space : AddrSpace) (dst : RegName) (src : Operand)
  | barSync (barrierId : Nat)
  | unsupported (opcode : String) (modifiers : Array String) (operands : Array Operand)
```

Parser Design

- Implemented with Lean's `Std.Internal.Parsec.String`.
- Tracks line and column on parse failure.
- Handles whitespace, line comments, block comments, and pragmas.
- Parses module directives, declarations, labels, instructions, and kernels.

Parser Instruction Coverage

- Arithmetic: `add`, `sub`, `mul.lo`, `mul.wide.s32`, `mad.lo`, `fma.rn`.
- Predicate ops: `setp`, `or.pred`, `and.pred`, `xor.pred`.
- Memory: `ld.<space>.<ty>`, `st.<space>.<ty>`.
- Address: `cvta.to.<space>`, `isspacep.<space>`.
- Control: labels, `bra`, guarded `@p bra`, `exit`/`ret`.

Parser Block Construction

- Statements are parsed as instructions, terminators, guarded branches, or no-ops.
- Guarded branches split the current statement stream into a block ending in `cbra`.
- Fallthrough blocks receive generated labels.
- Implicit fallthroughs are linked to the next parsed block or to exit.

Lowering Strategy

- Unchecked lowering exists for direct syntax-to-IR mapping.
- Checked lowering is the important path.
- Checked lowering threads a `Typing.TypeEnv`.
- Each instruction checks operands, supported types, guards, labels, params, and shared symbols.

Lowering Error Model

- `unknownReg`, `unknownPred`, `unknownParam`, `unknownShared`.
- `unknownSymbol`, `unknownBlock`.
- `typeMismatch`, `unsupportedType`, `unsupportedOp`, `unsupportedModifier`.
- This is crucial for making unsupported PTX fail loudly at the boundary.

Checked Lowering Examples

- `mov.u32` lowers to `assignReg`.
- `add.u32` lowers to `assignReg` with `RValue.binop add`.
- `setp.ge.s32` lowers to `assignPred`.
- `ld.global.s32` lowers to `load` with a typed global address.
- `st.global.s32` lowers to `store`.

Lowering Skeleton

```
def lowerInstrChecked? (env : Typing.TypeEnv) :
    Instr -> LowerM (CLean.Instr x Typing.TypeEnv)
  | .mov ty dst src => do
      expectOperandType env ty src
      let src <- lowerOperandCheckedAs? env ty src
      pure (.assignReg dst src, { env with regs := env.regs.insert dst ty })
  | .ld space ty dst addr => do
      ensureCodecType ty
      let addrTy <- addressOperandType? env space addr
      ...
```

Note: the actual file uses Lean product syntax; this slide uses `x` visually for readability.

Kernel Environment Lowering

- Parameters are laid out with `lowerParamsChecked?`.
- Shared declarations are laid out with `lowerSharedsChecked?`.
- Blocks are lowered into a `HashMap BlockLabel Block`.
- Entry label is checked against the block-label map.

Parser/Lowering Bridge

- `BridgeError` distinguishes parse errors from lowering errors.
- `parseAndLowerKernel?` parses one kernel and checked-lowers it.
- `parseAndLowerModule?` checks every kernel in a parsed module.
- Boolean wrappers power examples: `parseAndLowerKernelOk?`, `parseAndLowerModuleOk?`.

Proof Bridge Status

- `Proof/Bridge.lean` defines `LoweredKernelWf` and `LoweredModuleWf`.
- admitted: `lowerKernelEnvChecked_wf`.
- admitted: `parseAndLowerKernel?_sound`.
- admitted: `parseAndLowerModule?_sound`.
- These are clean theorem surfaces, but not yet fully proved.

Example Strategy

- Start with tiny semantic IR kernels: assign, copy, barrier.
- Then test manually built PTX AST lowering.
- Then parse PTX text and lower it.
- Finally execute parsed/lowered kernels with `runN` and boolean postconditions.

Semantic Examples

- Toy assign: one lane writes `r1 = 7` and terminates.
- Toy copy: load global `u32`, store to another global offset, terminate.
- Toy barrier: barrier releases under expected count 1, then assignment and termination.
- These examples use `cstep`, `runN`, and `native_decide`.

Lowering Examples

- Direct PTX AST instructions are lowered and compared by reflexivity.
- Checked lowering rejects use-before-declare.
- Parsed/lowered PTX kernels run under the semantic driver.
- Coverage includes assign, add, copy, param copy, barrier, and shared memory.

Executable Validation Philosophy

- Concrete examples do not prove parametric correctness.
- They are still valuable because they exercise the actual parser, lowering, and semantics.
- `native_decide` catches many integration bugs quickly.
- The proof architecture then explains what remains beyond testing.

Saxpy Target

The saxpy-style integer kernel computes:

$$r[i] = x[i] * \alpha + y[i]$$

for active lanes whose thread index is below `n`.

In this repo, the data values are `s32`; the postcondition uses 32-bit signed wrapping through `s32Wrap`.

Saxpy PTX Shape

- Parameters: `n`, `alpha`, `x` pointer, `y` pointer, `r` pointer.
- Loads parameter values from `.param`.
- Computes `threadIdx = ctaid.x * ntid.x + tid.x`.
- Branches to return if `threadIdx >= n`.
- Otherwise loads `x[i]` and `y[i]`, computes `x[i] * alpha + y[i]`, stores `r[i]`.

Saxpy PTX Core

```
ld.param.u32 %r2, [saxpyKernel_param_0];
ld.param.s32 %r6, [saxpyKernel_param_1];
ld.param.u64 %rd1, [saxpyKernel_param_2];
...
mad.lo.s32 %r1, %r3, %r4, %r5;
setp.ge.s32 %p1, %r1, %r2;
@%p1 bra $L__BB0_2;
...
ld.global.s32 %r7, [%rd6];
ld.global.s32 %r8, [%rd8];
mad.lo.s32 %r9, %r7, %r6, %r8;
st.global.s32 [%rd10], %r9;
```

Saxpy Parsing and Lowering

- `saxpyKernelText` stores the PTX source.
- `PTX.parseAndLowerKernelOk? saxpyKernelText = true` is checked by `native_decide`.
- `PTX.lowerKernelSupported? saxpyKernel = true` is also checked.
- This validates that the current parser and checked-lowering subset covers this kernel.

Saxpy Initial State

- `saxpyXBase = 0`, `saxpyYBase = 128`, `saxpyRBase = 256`.
- Parameter memory stores `n`, `alpha`, and three base pointers.
- Global memory stores input vectors `xs` and `ys`.
- One CTA and one warp are populated; active mask is `activeMaskPrefix n`.

saxpyStateFor

```
def saxpyStateFor (n : Nat) (alpha : Int) (xs ys : List Int) : State :=
  { kernelEnv := PTX.lowerKernelEnvCheckedD saxpyKernel
    global := { bytes := saxpyGlobalBytesFor xs ys }
    param := { bytes := saxpyParamBytesFor n alpha }
    ctas := ({} : Std.HashMap CTAId CTAState).insert 0
      { warps := ({} : Std.HashMap WarpId WarpState).insert 0
          (saxpyWarpFor n) } }
```

Saxpy Postcondition

- `saxpyExpectedAt alpha xs ys i` computes wrapped `xs[i] * alpha + ys[i]`.
- `saxpyPost base n alpha xs ys st` requires every index below `n` to match expected global memory.
- It is built from `vectorS32Post`.
- This is the parametric final property for saxpy.

Saxpy Spec Helpers

```
def saxpyExpectedAt (alpha : Int) (xs ys : List Int) (i : Nat) : Int :=
  s32Wrap (listIntGetD xs i * alpha + listIntGetD ys i)

def saxpyPost (base n : Nat) (alpha : Int) (xs ys : List Int)
    (st : State) : Prop :=
  vectorS32Post base n (saxpyExpectedAt alpha xs ys) st
```

Concrete Saxpy Status

- File header documents an axiom-clean concrete run for `n = 4`.
- Expected output: `[12, 24, 36, 48]` for `alpha = 2`, `xs = [1,2,3,4]`, `ys = [10,20,30,40]`.
- The current visible file focuses on the general theorem path and the admitted symbolic execution lemma.
- Running `CLean/Examples/Saxpy.lean` currently succeeds with a warning that `saxpy_runN_satisfies_post` uses `sorry`.

Saxpy Single-Warp Lemma

- `saxpyStateFor_isSingleWarp` is proved.
- It unfolds the state construction and proves any successful warp lookup must be CTA 0, warp 0.
- This is the entry point into the single-warp determinism machinery.
- It is specific to the state constructor, not an axiom about the kernel.

Saxpy Proof Reduction

The general `saxpy_partial_correct` proof has a meaningful skeleton:

1. Initial state is `IsSingleWarp`.
2. `step?` preserves `IsSingleWarp`.
3. Any terminal relational final equals `runN K init`.
4. A symbolic execution theorem supplies a terminal `runN J init` satisfying the post.
5. Stuck-stability equates the two terminal `runN` endpoints.

Remaining Saxpy Bottleneck

The key remaining theorem is:

`saxpy_runN_satisfies_post`

It must show that for every supported launch shape `n <= 32` with matching input lengths, there exists fuel `K` such that `runN K init` is final and satisfies `saxpyPost`.

What the Saxpy RunN Theorem Requires

- A straight-line symbolic execution of the saxpy body.
- Per-lane reasoning for thread index, branch predicate, pointer offsets, loads, multiply-add, and store.
- Read-after-write memory lemmas for each lane's output slot.
- Lane-disjointness proof: lane `i` writes only output address `rBase + i * 4`.
- Lift from per-lane stores to vector postcondition.

Saxpy Early-Exit Case

- For lanes with `i >= n`, `setp.ge.s32` sets the branch predicate.
- The guarded branch takes them to return.
- These lanes should not write output.
- The postcondition only quantifies `i < n`, so inactive/out-of-range behavior is framed away.

Saxpy Active-Lane Case

- For lanes with `i < n`, the branch predicate is false.
- The lane converts parameter pointers to global addresses.
- It computes byte offset `i * 4`.
- It reads `x[i]` and `y[i]`.
- It writes `s32Wrap (x[i] * alpha + y[i])` to `r[i]`.

Saxpy Memory Argument

- Input and output regions are separated by base constants 0, 128, and 256.
- For `n <= 32`, each vector occupies at most 128 bytes.
- Output slots `rBase + i * 4` are pairwise distinct.
- The write/read lemmas already cover global `s32` stores, but the vector-wide frame/lift is still needed.

What Is Left Before saxpy Is Honest

- Fill remaining `Proof/Lemmas.lean` frame and preservation holes.
- Strengthen `Proof/LaneDecomposition.lean` from shape statements to pointwise lane execution.
- Prove the active-lane symbolic execution sequence for saxpy.
- Prove lane-disjoint store addresses.
- Package the postcondition proof and remove `saxpy_runN_satisfies_post`'s `sorry`.

Saxpy Progress Report

Component	Status
PTX source parses	executable
Checked lowering accepts kernel	executable
State constructor and memory layout	implemented
Single-warp initial-state lemma	proved
Determinism reduction for partial correctness	proved, with dependencies
Symbolic run theorem	admitted
Total correctness theorem	depends on admitted symbolic run

Known Semantic Limitations

- General warp scheduling and multi-CTA interleavings are not solved.
- Non-uniform conditional branches currently fail rather than model divergence/reconvergence.
- Warp ops, atomics, and MMA are represented but mostly not implemented in `stepInstr?`.
- Floating-point semantics use Lean `Float`; this is not bit-exact PTX f32.

Known Proof Limitations

- Several frame and preservation theorems are currently admitted.
- Parser/lowering soundness is stated but not yet proved.
- Lane decomposition is architected but not strong enough for saxpy yet.
- Matmul general correctness is intentionally admitted pending loop-invariant proofs.

Why the Existing Sorries Are Manageable

- They are named and localized rather than hidden behind broad trusted theorems.
- Many have mechanical proof shapes: unfold executable function, split cases, apply update frame lemmas.
- The saxpy remaining theorem is kernel-specific and can be decomposed by lane and instruction.
- The proof surface makes it clear what `\#print axioms` should report.

Recommended Next Milestone

1. Finish `stepInstr?_preserves_kernelEnv` and wf preservation.
2. Finish barrier frame/support lemmas or restrict saxpy path to no-barrier dependencies where possible.
3. Prove a strong `applyToLaneIds?` lane-in/lane-not-in theorem sufficient for multi-lane register updates.
4. Prove saxpy active-lane straight-line computation.
5. Remove `saxpy_runN_satisfies_post`'s `sorry`.

Longer-Term Path

- Prove parser/lowering well-formedness.
- Generalize from single-warp to scheduler-independent multi-CTA reasoning.
- Model divergence/reconvergence if needed for real PTX kernels.
- Replace float semantics with bit-precise f32/f64 if hardware equivalence is the goal.
- Instantiate `CountedLoopSpec` for matmul's unrolled and remainder loops.

What This Deck Claims

- CLean already has a functioning semantic pipeline from parsed PTX to executable IR.
- Concrete examples are running through the actual semantics.
- The theorem architecture for saxpy is in place.
- The current blocking work is well-scoped proof engineering, not redesign.
- Scaling beyond saxpy will require stronger lane and loop infrastructure.

Files to Discuss Live

- `CLean/Core/IR.lean`: semantic instruction vocabulary.
- `CLean/Semantics/Helpers.lean`: executable behavior.
- `CLean/Proof/Determinism.lean`: single-warp determinism bridge.
- `CLean/PTX/Parser.lean`: PTX ingestion.
- `CLean/PTX/Lowering.lean`: checked lowering.
- `CLean/Examples/Saxpy.lean`: case study and current proof boundary.

Discussion

The central advisor question is now strategic:

Next proof target

Should the immediate milestone be a fully honest saxpy proof under the current single-warp model, or should the project first strengthen the semantic model for divergence and scheduling before investing in kernel-specific proof?